

МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ
**"САНКТ-ПЕТЕРБУРГСКИЙ ПОЛИТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ ПЕТРА ВЕЛИКОГО"**
Институт компьютерных наук и технологий
Направление **02.03.01** : Математика и компьютерные науки

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ
«Автоматизированное тестирование»

Исполнитель: _____

Богдан Арсений Вячеславович
группа 5130201/20002

Руководитель: _____

Курочкин Михаил Александрович

« ____ » _____ 2025г

Санкт-Петербург, 2025

Содержание

Введение	3
1 Постановка задачи	4
2 Описание средств автоматизации тестирования	5
2.1 JUnit	5
2.2 Selenium WebDriver	6
2.3 TestNG	7
3 Задача 1	8
3.1 Подготовительные операции	8
3.2 Тесты функции sum	8
3.3 Тесты функции mult	9
3.4 Тесты функции div	11
3.5 Тесты функции sqrt	12
3.6 Общая структура тестов	13
3.7 Результаты тестирования	14
4 Задача 2	15
4.1 DriverSetup	16
4.1.1 Структура класса	16
4.2 Task1Test	17
4.2.1 Особенности реализации	17
4.3 Task2Test	18
4.3.1 Метод testTitle()	18
4.3.2 Метод testLogin()	18
4.3.3 Метод testElement()	19
4.4 Результаты тестирования	19
5 Заключение	21
Список литературы	22

Введение

Тестирование программного обеспечения - это процесс определения несоответствия поведения программного продукта заявленной спецификации.

Главная задача тестирования — обнаружить как можно больше дефектов, связанных с тем, что продукта не соответствует заявленным требованиям. При этом желательно проводить тестирование программного продукта эффективно с точки зрения временных затрат.

Автоматизированное тестирование - это метод проверки программного обеспечения на соответствие заявленной спецификации посредством использования специализированных инструментов и фреймворков для многократного выполнения однотипных проверок без участия человека. Преимущества автоматизированного тестирования:

- Автоматизированное тестирование надежнее ручного, так как снижает влияние человеческого фактора.
- Ручное тестирование не только отнимает больше времени, но и чревато внесением новых ошибок.
- Инструменты автоматизации помогают обнаруживать ошибки в программном продукте на ранних стадиях его разработки.
- Высокая производительность приложений: благодаря максимальному охвату тестированием автоматизированное тестирование обеспечивает высокое качество и производительность программного продукта.

Автоматизация играет важную роль в современной разработке ПО, особенно в крупных и долгосрочных проектах, где требуется поддерживать высокий уровень качества продукта.

1 Постановка задачи

В данной работе необходимо:

- Создать юнит-тесты для библиотеки `calculator.jar` с использованием фреймворка JUnit:
 - Разработать юнит-тесты для четырех методов библиотеки `calculator.jar` с использованием фреймворка JUnit.
 - Запустить тесты при помощи используемой среды разработки.
 - Продемонстрировать результаты выполнения тестов.
- Провести тестирование веб-сайта с использованием TestNG и Selenium WebDriver:
 - Написать два теста, проверяющих корректное отображение и работу страниц веб-сайта.
 - Организовать код тестов в соответствии с Java Code Convention.
 - Запустить тесты через конфигурационный файл TestNG suite xml.

2 Описание средств автоматизации тестирования

2.1 JUnit

JUnit — это фреймворк для написания и выполнения модульных тестов на языке программирования Java. Он позволяет разработчикам создавать и повторять тесты для проверки правильности работы кода.

Ключевые особенности JUnit

- **Аннотации:** JUnit предоставляет различные аннотации, такие как `@Test`, `@Before`, `@After`, `@BeforeAll` и `@AfterAll`, для задания и настройки тестов.
- **Поддержка параметризованных тестов:** С помощью аннотаций `@ParameterizedTest` и `@ValueSource` можно передавать параметры в тесты.
- **Интеграция с различными средами разработки:** JUnit легко интегрируется с большинством IDE, таких как IntelliJ IDEA, Eclipse и NetBeans.
- **Простота использования:** JUnit легко освоить, что делает его популярным среди программистов.

Процесс написания тестов

1. **Создание теста:** Написание тестового метода и добавление к нему аннотации `@Test`.
2. **Настройка тестов при помощи следующих аннотаций:**
 - `@BeforeEach`: Аннотированный метод будет запускаться перед каждым тестовым методом в тестовом классе.
 - `@AfterEach`: Аннотированный метод будет запускаться после каждого тестового метода в тестовом классе.
 - `@BeforeAll`: Аннотированный метод будет запущен перед всеми тестовыми методами в тестовом классе. Этот метод должен быть статическим.
 - `@AfterAll`: Аннотированный метод будет запущен после всех тестовых методов в тестовом классе. Этот метод должен быть статическим.
 - `@DisplayName`: Используется для предоставления любого настраиваемого отображаемого имени для тестового класса или тестового метода
 - `@Disable`: Он используется для отключения или игнорирования тестового класса или тестового метода из набора тестов.
 - `@Nested`: Используется для создания вложенных тестовых классов
 - `@Tag` Пометьте методы тестирования или классы тестов тегами для обнаружения и фильтрации тестов.
 - `@TestFactory` Отметить метод - это тестовая фабрика для динамических тестов.
3. **Запуск тестов:** Выполнение тестов с помощью встроенных инструментов IDE или командной строки.

2.2 Selenium WebDriver

Selenium - один из наиболее популярных фреймворков для автоматизации тестирования сайтов и веб-приложений. Инструмент имеет открытый исходный код.

Selenium поддерживается разными операционными системами (Mac, Linux, Windows) и большинством популярных браузеров (Firefox, IE, Chrome, браузеры Headless). Скрипты для фреймворка можно писать на Python, C#, PHP, Java и других языках программирования.

Selenium WebDriver — это инструмент для автоматизации взаимодействия с веб-браузерами. Он позволяет программно управлять браузером, имитируя действия пользователя (клики, ввод текста, навигацию и т. д.). WebDriver является частью экосистемы **Selenium** и поддерживает множество языков программирования, включая Java, Python, C# и JavaScript.

Основные методы WebDriver

WebDriver предоставляет множество методов для управления браузером:

- `get(url)` — открывает указанный URL.
- `findElement(By.locator)` — находит элемент на странице (по ID, XPath, CSS-селектору и др.).
- `click()`, `sendKeys(text)` — взаимодействие с элементами (клик, ввод текста).
- `navigate().back()`, `navigate().forward()` — управление историей браузера.
- `quit()` — закрывает браузер и освобождает ресурсы.

Преимущества и недостатки

- **Плюсы:**
 - Работает с Chrome, Firefox, Edge, Safari и другими браузерами.
 - API доступен для Java, Python, C#, Ruby и других языков.
 - Совместим с JUnit, TestNG, PyTest и другими фреймворками для тестирования.
 - Позволяет взаимодействовать с динамическими элементами веб-страниц (AJAX, JavaScript).
 - Запуск браузера без графического интерфейса для ускорения тестов.
 - Открытый исходный код.
 - Поддержка множества браузеров и операционных систем.
 - Большое сообщество и множество документации.
- **Минусы:**
 - Требуется установки отдельных драйверов для каждого браузера.
 - Может быть медленным для сложных сценариев.
 - Нет встроенного отчета о тестах (требуются дополнительные фреймворки).

Процесс взаимодействия с браузером при помощи WebDriver

1. Создание экземпляра WebDriver.
2. Открытие браузера при помощи экземпляра WebDriver.
3. Открытие тестируемого сайта по URL.
4. Проведение тестового сценария.
5. Закрытие браузера.

2.3 TestNG

TestNG — это фреймворк для автоматизированного тестирования программного обеспечения, написанный на языке Java. Он также как и JUnit предназначен для создания и запуска тестов для проверки функциональности и качества программных продуктов.

Хотя JUnit послужил вдохновением для TestNG, второй имеет ряд отличий и, в отличие от JUnit, работает для функционального и более высоких уровней тестирования.

TestNG также поддерживает параллельное выполнение тестов, что позволяет ускорить процесс тестирования и повысить производительность. Кроме того, фреймворк интегрируется с другими инструментами и системами, такими как Jenkins, Maven и Eclipse, что упрощает процесс тестирования и развертывания программного обеспечения.

3 Задача 1

Дано:

- Библиотечный файл calculator.jar.

Требуется:

- Создать java проект.
- в файле build.gradle прописать зависимости для JUnit фреймворка.
- Добавить в проект файл calculator.jar.
- Выбрать четыре метода из предоставленной библиотеки по своему усмотрению и написать для них юнит-тесты с использованием фреймворка JUnit:
 - Все подготовительные операции должны быть выполнены в методах с аннотацией @Before.
 - Все завершающие операции должны быть выполнены в методах с аннотацией @After.
 - Данные для тестов передавать через параметры с использованием аннотаций @ParameterizedTest и @ValueSource.
- Запустить тесты через интегрированную среду разработки (IDE).

3.1 Подготовительные операции

Инициализация переменной calculator производится единожды перед началом запуска тестов ссылкой на экземпляр класса Calculator. Далее этот статический объект будет использоваться каждой из тестовых функций. Делается это с целью снижения затрат памяти, а также ускорения процесса выполнения тестов, так как не затрачиваются дополнительные ресурсы на создание излишних экземпляров класса Calculator.

Вся логика подготовительных операций описана в классе CalculatorTest, код которого приведён в Приложении 1.

3.2 Тесты функции sum

Дано: функция sum класса Calculator. Функция принимает следующие параметры:

- Левый операнд: double a;
- Правый операнд: double b;

Функция возвращает тип double. Также из названия функции можно предположить, что её поведение должно соответствовать математической операции сложения.

Исходя из названия функции, типов параметров функции и возвращаемого типа, можно составить тесты, используя техники классов эквивалентностей и анализа граничных значений.

Классы эквивалентностей: Итоги тестирования классов эквивалентностей указаны в таблице 1.

Входное условие	Допустимые классы	Недопустимые классы
a	$a \in Double$	-
b	$b \in Double$	-

Таблица 1: Разбиение на классы эквивалентности

№	a	b	Ожидаемый результат	Фактический результат	Статус теста
1	1.23	1.75	С = 2,98 +- 0.0001 Завершение выполнения функции	С = 2,98 +- 0.0001 Завершение выполнения функции	Пройден

Таблица 2: Тесты для проверки классов эквивалентностей

Граничные значения:

- Верхние граничные значения: a = 1.7976931348623157E308, b = 1.7976931348623157E308
- Нижние граничные значения: a = -1.7976931348623157E308, b = -1.7976931348623157E308

Итоги тестирования классов эквивалентностей указаны в таблице 2.

№	a	b	Ожидаемый результат	Фактический результат	Статус теста
1	1.7976931348623157E308	1.75	Результат совпадает с оператором '+' Завершение выполнения функции	Результат совпадает с оператором '+' Завершение выполнения функции	Пройден
2	1.75	1.7976931348623157E308	Результат совпадает с оператором '+' Завершение выполнения функции	Результат совпадает с оператором '+' Завершение выполнения функции	Пройден
3	-1.7976931348623157E308	1.75	Результат совпадает с оператором '+' Завершение выполнения функции	Результат совпадает с оператором '+' Завершение выполнения функции	Пройден
4	1.75	-1.7976931348623157E308	Результат совпадает с оператором '+' Завершение выполнения функции	Результат совпадает с оператором '+' Завершение выполнения функции	Пройден

Таблица 3: Тесты для проверки граничных условий

3.3 Тесты функции mult

Дано: функция mult класса Calculator. Функция принимает следующие параметры:

- Левый операнд: double a;
- Правый операнд: double b;

Функция возвращает тип double. Также из названия функции можно предположить, что её поведение должно соответствовать математической операции умножения.

Исходя из названия функции, типов параметров функции и возвращаемого типа, можно составить тесты, используя техники классов эквивалентностей и анализа граничных значений.

Классы эквивалентностей:

Входное условие	Допустимые классы	Недопустимые классы
a	$a \in Double$	-
b	$b \in Double$	-

Таблица 4: Разбиение на классы эквивалентности

Итоги тестирования классов эквивалентностей указаны в таблице 5.

№	a	b	Ожидаемый результат	Фактический результат	Статус теста
1	1.23	1.75	Результат совпадает с оператором '*' Завершение выполнения функции	Результат совпадает с оператором '*' Завершение выполнения функции	Не пройден

Таблица 5: Тесты для проверки классов эквивалентностей

Граничные значения:

- Верхние граничные значения: $a = 1.7976931348623157E308$, $b = 1.7976931348623157E308$
- Нижние граничные значения: $a = -1.7976931348623157E308$, $b = -1.7976931348623157E308$

Итоги тестирования классов эквивалентностей указаны в таблице 6.

№	a	b	Ожидаемый результат	Фактический результат	Статус теста
1	1.7976931348623157E308	1.75	Результат совпадает с оператором '*' Завершение выполнения функции	Результат совпадает с оператором '*' Завершение выполнения функции	Пройден
2	1.75	1.7976931348623157E308	Результат совпадает с оператором '*' Завершение выполнения функции	Результат совпадает с оператором '*' Завершение выполнения функции	Пройден
3	-1.7976931348623157E308	1.75	Результат совпадает с оператором '*' Завершение выполнения функции	Результат совпадает с оператором '*' Завершение выполнения функции	Пройден
4	1.75	-1.7976931348623157E308	Результат совпадает с оператором '*' Завершение выполнения функции	Результат совпадает с оператором '*' Завершение выполнения функции	Пройден

Таблица 6: Тесты для проверки граничных условий

3.4 Тесты функции div

Дано: функция div класса Calculator. Функция принимает следующие параметры:

- Левый операнд: double a;
- Правый операнд: double b;

Функция возвращает тип double. Также из названия функции можно предположить, что её поведение должно соответствовать математической операции деления.

Исходя из названия функции, типов параметров функции и возвращаемого типа, можно составить тесты, используя техники классов эквивалентностей и анализа граничных значений.

Классы эквивалентностей: Итоги тестирования классов эквивалентностей указаны в таб-

Входное условие	Допустимые классы	Недопустимые классы
a	$a \in Double$	-
b	$b \in Double$	$b! = 0$

Таблица 7: Разбиение на классы эквивалентности

лице 8.

№	a	b	Ожидаемый результат	Фактический результат	Статус теста
1	1.23	1.75	Результат совпадает с оператором '/' Завершение выполнения функции	Результат совпадает с оператором '*' Завершение выполнения функции	Пройден
2	1.23	0	Выброшена ошибка ArithmeticException	Выброшена ошибка ArithmeticExceptio	Пройден

Таблица 8: Тесты для проверки классов эквивалентностей

Граничные значения:

- Верхние граничные значения: $a = 1.7976931348623157E308$, $b = 1.7976931348623157E308$
- Нижние граничные значения: $a = -1.7976931348623157E308$, $-b = 1.7976931348623157E308$

Итоги тестирования классов эквивалентностей указаны в таблице 9.

№	a	b	Ожидаемый результат	Фактический результат	Статус теста
1	1.797 6931348 623157 E308	1.75	Результат совпадает с оператором '/' Завершение выполнения функции	Результат совпадает с оператором '/' Завершение выполнения функции	Пройден
2	1.75	1.797 6931348 623157 E308	Результат совпадает с оператором '/' Завершение выполнения функции	Результат совпадает с оператором '/' Завершение выполнения функции	Пройден
3	-1.797 6931348 623157 E308	1.75	Результат совпадает с оператором '/' Завершение выполнения функции	Результат совпадает с оператором '/' Завершение выполнения функции	Пройден
4	1.75	-1.797 6931348 623157 E308	Результат совпадает с оператором '/' Завершение выполнения функции	Результат совпадает с оператором '/' Завершение выполнения функции	Пройден

Таблица 9: Тесты для проверки граничных условий

3.5 Тесты функции sqrt

Дано: функция sqrt класса Calculator. Функция принимает следующие параметры:

- Значение: double a;

Функция возвращает тип double. Также из названия функции можно предположить, что её поведение должно соответствовать математической операции взятия квадратного корня.

Исходя из названия функции, типов параметров функции и возвращаемого типа, можно составить тесты, используя техники классов эквивалентностей и анализа граничных значений.

Классы эквивалентностей: Итоги тестирования классов эквивалентностей указаны в таб-

Входное условие	Допустимые классы	Недопустимые классы
a	$a \in Double, a \geq 0$	$a < 0$

Таблица 10: Разбиение на классы эквивалентности

лице 8.

№	a	Ожидаемый результат	Фактический результат	Статус теста
1	4	Результат совпадает с оператором '*' Завершение выполнения функции	Результат совпадает с оператором '*' Завершение выполнения функции	Пройден
3	-10	Выброшена ошибка ArithmeticException	Был возвращён результат извлечения квадратного корня от модуля числа	Не пройден

Таблица 11: Тесты для проверки классов эквивалентностей

Граничные значения:

- Верхние граничные значения: $a = 1.7976931348623157E308$.
- Нижние граничные значения: $a = -1.7976931348623157E308$.

Итоги тестирования классов эквивалентностей указаны в таблице 9.

№	a	Ожидаемый результат	Фактический результат	Статус теста
1	1.797 6931348 623157 E308	Результат совпадает с оператором '/' Завершение выполнения функции	Результат совпадает с оператором '/' Завершение выполнения функции	Пройден
2	-1.797 6931348 623157 E308	Результат совпадает с оператором '/' Завершение выполнения функции	Результат совпадает с оператором '/' Завершение выполнения функции	Пройден

Таблица 12: Тесты для проверки граничных условий

3.6 Общая структура тестов

Тестовые классы `DivLongTest`, `MultDoubleTest`, `SqrtDoubleTest` и `SumDoubleTest` проверяют функциональность класса `Calculator` с использованием фреймворка JUnit 5. Каждый из этих классов наследуется от класса `CalculatorTest`. В классе `CalculatorTest` описан метод с аннотацией `@BeforeAll`, отвечающий за инициализацию переменной типа `Calculator` перед запуском всех тестов. Также он содержит константу `DELTA` равную 0.0001. Эта константа выступает в качестве погрешности для сравнения чисел с плавающей точкой. Исходный код каждого из классов можно найти в Приложении 1-5

Когда как тестовые классы содержат следующие элементы:

- Параметризованные тесты с использованием `@ParameterizedTest`
- Различные источники данных: `@ValueSource` и `@CsvSource`

При помощи аннотации `@ParameterizedTest` указывается, что тестовая функция является параметризованной, а в аннотации `@CsvSource` происходит перечисление входных параметров через запятую.

3.7 Результаты тестирования

✓ ✗ <default package>	41 ms
✓ DivLongTest	30 ms
✓ testGeneralCorrectness(double, double)	30 ms
✓ [1] 1.23, 1.75	27 ms
✓ [2] 1.23, 0	1 ms
✓ [3] 1.7976931348623157E308, 1.75	1 ms
✓ [4] 1.75, -1.7976931348623157E308	
✓ [5] 1.7976931348623157E308, 1.75	
✓ [6] 1.75, -1.7976931348623157E308	1 ms
✗ MultDoubleTest	6 ms
✗ testMult(double, double)	6 ms
✗ [1] 1.23, 1.75	6 ms
✓ [2] 1.7976931348623157E308, 1.75	
✓ [3] 1.75, 1.7976931348623157E308	
✓ [4] -1.7976931348623157E308, 1.75	
✓ [5] 1.75, -1.7976931348623157E308	
✗ testSqrt(double)	4 ms
✓ [1] 4.0	2 ms
✗ [2] -10.0	2 ms
✓ [3] 1.7976931348623157E308	
✓ [4] 4.9E-324	
✓ SumDoubleTest	1 ms
✓ testSum(double, double)	1 ms
✓ [1] 1.23, 1.75	
✓ [2] 1.7976931348623157E308, 1.75	1 ms
✓ [3] 1.75, 1.7976931348623157E308	
✓ [4] -1.7976931348623157E308, 1.75	
✓ [5] 1.75, -1.7976931348623157E308	

Рис. 1: Результат запуска тестов

При помощи фреймворка JUnit были разработаны автоматизированные тесты для библиотеки Calculator. Эти тесты производят проверку 4-х функций из этой библиотеки. Из всех 18 тестов не прошли 2 теста, один относится к функции sqrt, другой к функции mult.

4 Задача 2

Дано: Драйвер для работы с браузером Google Chrome, ссылка на тестовый сайт: <https://jdi-testing.github.io/jdi-light/index.html>

Для выполнения задачи 2 требуется:

- Подключить в проекте зависимости Selenium и TestNG, а также добавить драйвер для работы с браузером Chrome в проект.
- Создать два теста на языке Java с использованием Selenium WebDriver и фреймворка TestNG для проверки корректности отображения страниц сайта <https://jdi-testing.github.io/jdi-light/index.html> в браузере Chrome.
- Тесты должны соответствовать стандартам Java Code Convention и запускаться с помощью TestNG suite xml.

Первый и второй тест должны проигрывать 1 и 2 тестовый сценарий соответственно. Тестовые сценарии представлены на рисунках 2-3.

Упражнение 1

Для этого упражнения используйте SoftAsserts.

#	Шаг	Данные	Ожидаемый результат
1	Open test site by URL	https://jdi-testing.github.io/jdi-light/index.html	Test site is opened
2	Assert Browser title	"Home Page"	Browser title equals "Home Page"
3	Perform login	username: Roman pass: Jdi1234	User is logged
4	Assert Username is logged in	"ROMAN IOVLEV"	Name is displayed and equals to expected result
5	Assert that there are 4 items on the header section are displayed and they have proper texts	"HOME", "CONTACT FORM", "SERVICE", "METALS & COLORS"	Menu buttons are displayed and have proper texts
6	Assert that there are 4 images on the Index Page and they are displayed	4 images	Images are displayed
7	Assert that there are 4 texts on the Index Page under icons and they have proper text	4 texts below of each image	Texts are displayed and equal to expected
8	Assert that there is the iframe with "Frame Button" exist		The iframe exists
9	Switch to the iframe and check that there is "Frame Button" in the iframe		The "Frame Button" exists
10	Switch to original window back		Driver has focus on the original window
11	Assert that there are 5 items in the Left Section are displayed and they have proper text	"Home", "Contact form", "Service", "Metals & Colors", "Elements packs"	Left section menu items are displayed and have proper text
12	Close Browser		Browser is closed

Рис. 2: Тестовый сценарий 1

#	Шаг	Данные	Ожидаемый результат
1	Open test site by URL	https://jdi-testing.github.io/jdi-light/index.html	Test site is opened
2	Assert Browser title	"Home Page"	Browser title equals "Home Page"
3	Perform login	username: Roman pass: Jdi1234	User is logged
4	Assert User name in the left-top side of screen that user is logged	ROMAN IOVLEV	Name is displayed and equals to expected result
5	Open through the header menu Service -> Different Elements Page		Page is opened
6	Select checkboxes	Water, Wind	Elements are checked
7	Select radio	Selen	Element is checked
8	Select in dropdown	Yellow	Element is selected
9	Assert that - for each checkbox there is an individual log row and value is corresponded to the status of checkbox - for radio button there is a log row and value is corresponded to the status of radio button - for dropdown there is a log row and value is corresponded to the selected value.		Log rows are displayed and - checkbox name and its status are corresponding to selected - radio button name and its status is corresponding to selected - dropdown name and selected value is corresponding to selected
10	Close Browser		Browser is closed

Рис. 3: Тестовый сценарий 2

4.1 DriverSetup

Первоначально требовалось настроить класс `DriverSetup`. Он предназначен для инициализации и настройки `WebDriver` в тестовом проекте. `WebDriver` предоставляет API для:

- Настройки и запуска браузера Chrome
- Авторизации на тестовом сайте
- Корректного завершения работы браузера

4.1.1 Структура класса

Метод `setup()`

Он помечен аннотацией `@BeforeTest`, благодаря этому он запускается перед каждым тестом.

Метод производит следующие действия

1. Устанавливает путь к драйверу Chrome.
2. Настраивает HTTP-клиент.
3. Создает экземпляр `ChromeDriver`.
4. Разворачивает окно браузера на весь экран.
5. Открывает тестовую страницу в браузере.
6. Выполняет авторизацию.

```

1  @BeforeTest
2  public static void setup() {
3      System.setProperty("webdriver.chrome.driver",
4          "src/test/resources/chromedriver.exe");
5      System.setProperty("webdriver.http.factory", "jdk-http-client");
6
7      driver = new ChromeDriver();

```

```

8     driver.manage().window().maximize();
9
10    driver.navigate().to("https://jdi-testing.github.io/jdi-light/index.
        html");
11
12    driver.findElement(By.cssSelector("...")).click();
13    driver.findElement(By.id("name")).sendKeys("Roman");
14    driver.findElement(By.id("password")).sendKeys("Jdi1234");
15    driver.findElement(By.id("login-button")).click();
16 }

```

Метод exit()

Этот метод выполняется после выполнения всех тестов. Он производит закрытие браузера при помощи `driver.close()`.

```

1  @AfterTest
2  public static void exit() {
3      driver.close();
4  }

```

4.2 Task1Test

Класс `Task1Test` наследуется от `DriverSetup` и содержит автоматизированные тесты для веб-страницы с использованием Selenium WebDriver и TestNG. Код класса представлен в приложении 6. Основные особенности:

- Использует `SoftAssert` для накопления проверок.
- Производит комплексную проверку элементов интерфейса.
- Работает с `iframe` - HTML-элементом (`<iframe>`), который позволяет встраивать другую веб-страницу или документ внутрь текущей.
- Проверяет текстовое содержимое элементов.

Этот тестовый класс производит следующие проверки:

Проверки роверки

Компонент пользовательского интерфейса	Описание
Заголовок страницы	Проверяет что title страницы соответствует "Home Page"
Имя пользователя	Проверяет отображение имени пользователя "ROMAN IOVLEV"
Заголовки в шапке страницы	Проверяет 4 элемента навигации и их текстовые заголовки
Изображения	Проверяет наличие и видимость 4 изображений
Тексты под иконками	Проверяет 4 текстовых блока и их текстовое наполнение
iframe	Проверяет наличие iframe и кнопки внутри него
Левое меню	Проверяет 5 пунктов меню с правильными текстами

4.2.1 Особенности реализации

- Использование `SoftAssert` позволяет выполнить все проверки, даже если некоторые из них не прошли

- Для поиска элементов применяются различные стратегии:
 - По ID (`By.id`)
 - По CSS селекторам (`By.cssSelector`)
 - По имени класса (`By.className`)
- Переключение между вложенной веб-страницей (`iframe`) и основным сайтом

4.3 Task2Test

Тестовый класс `Task2Test` наследуется от `DriverSetup` и содержит автоматизированные тесты для проверки взаимодействия с элементами веб-интерфейса. Основные особенности:

- Тесты охватывают проверку заголовка страницы, процесс авторизации, взаимодействие с чекбоксами, радиокнопками и элементами выпадающих списков, а также правильность отображения логов, соответствующих этим действиям.
- Проверка логов изменений состояния элементов..

4.3.1 Метод `testTitle()`

```

1  @Test
2  public void testTitle() {
3      driver.get("https://jdi-testing.github.io/jdi-light/index.html");
4      assertEquals(driver.getTitle(), "HomePage");
5  }
```

- Проверяет заголовок страницы
- Использует простой `assert` для проверки
- Перенаправляет на тестовую страницу

4.3.2 Метод `testLogin()`

```

1  @Test
2  public void testLogin() {
3      driver.findElement(By.id("user-icon")).click();
4      driver.findElement(By.id("name")).sendKeys("Roman");
5      driver.findElement(By.id("password")).sendKeys("Jdi1234");
6      driver.findElement(By.id("login-button")).click();
7      assertEquals(driver.findElement(By.id("user-name")).getText(), "ROMAN
      IOVLEV");
8  }
```

- Выполняет процесс авторизации
- Проверяет отображение имени пользователя после входа
- Использует поиск элементов по ID

4.3.3 Метод testElement()

Самый комплексный тест, состоящий из нескольких частей:

1. Навигация на страницу с чекбоксами, радиокнопками и выпадающими списками:

```
1 driver.findElement(By.cssSelector("...")).click();
2 driver.findElement(By.xpath("/html/...")).click();
```

2. Активация чекбоксов "Water" и "Wind"

```
1 List<WebElement> checkboxes = driver.findElements(By.className("label -
checkbox"));
2 checkboxes.stream().filter(
3     (checkbox) -> checkbox.getText().equals("Water") || checkbox.
        getText().equals("Wind")
4 ).forEach(WebElement::click);
```

3. Активация радиокнопки "Selen"

```
1 List<WebElement> radios = driver.findElements(By.className("label-radio")
);
2 radios.stream().filter((button) -> button.getText().equals("Selen")).
    findFirst().get().click();
```

4. Выбор опции "Yellow" из выпадающего списка

```
1 List<WebElement> options = driver.findElements(By.tagName("option"));
2 options.stream().filter((option) -> option.getText().equals("Yellow")).
    findFirst().get().click();
```

5. Проверка логов: последним шагом производится проверка наличия и соответствия логов каждому выполненному действию на странице.

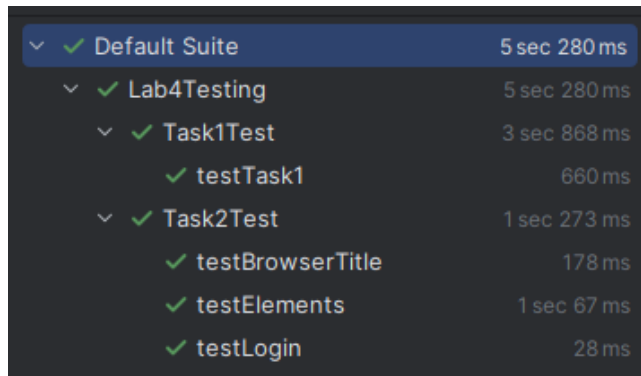
```
1 String logsText = driver.findElement(By.cssSelector("ul.panel-body-list.
logs")).getText();
2 String[] logLinesArray = logsText.split("\n");
3 List<String> logLines = Arrays.stream(logLinesArray).map(log -> log.
    substring(logIndexStart)).toList();
4
5 // Ожидаемые записи в логах
6 List<String> expectedLogEntries = List.of(
7     " Colors: value changed to Yellow",
8     " metal: value changed to Selen",
9     " Wind: condition changed to true",
10    " Water: condition changed to true"
11 );
12
13 // Проверить соответствие логов
14 assertEquals(logLines, expectedLogEntries);
```

4.4 Результаты тестирования

В ходе выполнения работы №2 была произведена успешная проверка отображения компонентов пользовательского интерфейса, а также взаимодействия с компонентами сайта. Благодаря использованию Selenium WebDriver и TestNG удалось настроить работу с браузером Chrome во время выполнения тестов, а также позволило доступным образом получить доступ к компонентам пользовательского интерфейса по их ID, по CSS селекторам, а также по имени класса. Также был рассмотрен механизм мягких утверждений, что

позволило провести комплексную проверку пользовательского интерфейса сайта в рамках одного запуска теста Task1Test.

Помимо этого тестирование проверяет текст заголовка страницы, наличие и текст пунктов меню, наличие изображений изображений, текста под картинками, наличие iframe и элементов в левом меню экрана. В том числе по окончании проведённых тестов производится освобождение захваченных ресурсов, а именно — закрытие браузера.



✓ Default Suite	5 sec 280 ms
✓ Lab4Testing	5 sec 280 ms
✓ Task1Test	3 sec 868 ms
✓ testTask1	660 ms
✓ Task2Test	1 sec 273 ms
✓ testBrowserTitle	178 ms
✓ testElements	1 sec 67 ms
✓ testLogin	28 ms

Рис. 4: Результаты тестирования

5 Заключение

В ходе выполнения лабораторной работы были изучены основные аспекты автоматизированного тестирования программного обеспечения. Были получены навыки написания и выполнения юнит-тестов с использованием фреймворка JUnit в результате разработки их для Java библиотеки `calculator.jar`. В результате проведённого тестирования при помощи JUnit было найдено две ситуации, в которых поведение тестируемых функций отличалось от ожидаемой. В том числе были получены навыки тестирования веб-страниц с помощью WebDriver и TestNG в ходе выполнения второй работы.

Результаты тестирования показали, что хоть автоматизированное тестирование обладает более высокой производительностью по сравнению с ручным тестированием, ручное тестирование сохраняет свою значимость. Оно позволяет выявить проблемы, которые могут быть не рассмотрены при автоматизации. В том числе, автоматические тесты также нужно проверять, ведь они являются результатом работы программистов. Таким образом, сочетание этих методов обеспечит лучшее качество программного обеспечения, чем в случае использования этих методов по отдельности.

Полученные навыки и рассмотренные фреймворки JUnit, Selenium, а также TestNG могут быть в дальнейшем применены в реальных проектах с целью времени тестирования программного продукта.

Список литературы

- [1] Майерс, Г. Искусство тестирования программ. – Санкт-Петербург: Диалектика, 2012. – С. 272.

Приложение 1. Исходный код CalculatorTest

```
1  class CalculatorTest {
2      protected static Calculator calculator;
3      protected static final double DELTA = 0.0001;
4
5      @BeforeAll
6      static void setUp() {
7          calculator = new Calculator();
8          System.out.println("Initializing Calculator instance before
9              tests");
10     }
11 }
```

Приложение 2. Исходный код SumDoubleTest

```
1  class SumDoubleTest extends CalculatorTest {
2
3      @ParameterizedTest
4      @CsvSource({
5          "1.23, 1.75",
6          "1.7976931348623157E308, 1.75",
7          "1.75, 1.7976931348623157E308",
8          "-1.7976931348623157E308, 1.75",
9          "1.75, -1.7976931348623157E308",
10     })
11     void testSum(double a, double b) {
12         assertEquals(a + b, calculator.sum(a, b), DELTA);
13     }
14 }
```

Приложение 3. Исходный код SqrtDoubleTest

```
1  class SqrtDoubleTest extends CalculatorTest {
2
3      @ParameterizedTest
4      @ValueSource(doubles = {4.0, -10, Double.MAX_VALUE, Double.MIN_
5          VALUE})
6      void testSqrt(double a) {
7          assertEquals(Math.sqrt(a), calculator.sqrt(a), DELTA);
8      }
9  }
```

Приложение 4. Исходный код MultDoubleTest

```
1  class MultDoubleTest extends CalculatorTest {
2
3      @ParameterizedTest
4      @CsvSource({
5          "1.23, 1.75",
```

```

6         "1.7976931348623157E308, 1.75",
7         "1.75, 1.7976931348623157E308",
8         "-1.7976931348623157E308, 1.75",
9         "1.75, -1.7976931348623157E308",
10    })
11    void testMult(double a, double b) {
12        assertEquals(a * b, calculator.mult(a, b), DELTA);
13    }
14 }

```

Приложение 5. Исходный код DivLongTest

```

1  class DivLongTest extends CalculatorTest {
2      @ParameterizedTest
3      @CsvSource({
4          "1.23, 1.75",
5          "1.23, 0",
6          "1.7976931348623157E308, 1.75",
7          "1.75, -1.7976931348623157E308",
8          "1.7976931348623157E308, 1.75",
9          "1.75, -1.7976931348623157E308"
10     })
11     void testGeneralCorrectness(double a, double b) {
12         assertEquals(a / b, calculator.div(a, b));
13         System.out.println(a/b);
14     }
15 }

```

Приложение 6. Исходный код Task1Test

```

1
2  import org.openqa.selenium.By;
3  import org.openqa.selenium.WebElement;
4  import org.testng.annotations.Test;
5  import org.testng.asserts.SoftAssert;
6
7  import java.util.List;
8
9  public class Task1Test extends DriverSetup {
10
11      @Test
12      public void testTask1() {
13          SoftAssert softAssert = new SoftAssert();
14
15          // 2. Assert browser title
16          softAssert.assertEquals(driver.getTitle(), "Home Page");
17
18          // 4. Assert Username is logged in
19          softAssert.assertEquals(driver.findElement(By.id("user-name")).
20                                  getText(), "ROMAN IOVLEV");
21
22          // 5. Assert that there are 4 items on the header section are
23              displayed, and they have proper texts
24          List<WebElement> headerItems = driver.findElements(By.
25              cssSelector("ul.uui-navigation.nav > li"));

```

```

23     softAssert.assertEquals(headerItems.size(), 4);
24
25     String[] expectedHeaderTexts = {"HOME", "CONTACT FORM", "SERVICE
26         ", "METALS & COLORS"};
27
28     for (int i = 0; i < headerItems.size(); i++) {
29         softAssert.assertTrue(headerItems.get(i).isDisplayed());
30         // Add an assertion to compare text
31         softAssert.assertEquals(headerItems.get(i).getText(),
32             expectedHeaderTexts[i]);
33         // Compare expected and actual text
34         softAssert.assertTrue();
35         softAssert.assertEquals(headerItems.get(i).getText(),
36             expectedHeaderTexts[i]);
37     }
38
39     // 6. Assert that there are 4 images on the Index Page, and they
40     // are displayed
41     List<WebElement> images = driver.findElements(By.cssSelector(".
42         benefit-icon > span"));
43     softAssert.assertEquals(images.size(), 4);
44     for (WebElement image : images) {
45         softAssert.assertTrue(image.isDisplayed());
46     }
47
48     // 7. Assert that there are 4 texts on the Index Page under
49     // icons, and they have proper text
50     List<WebElement> texts = driver.findElements(By.className("
51         benefit-txt"));
52     softAssert.assertEquals(texts.size(), 4);
53
54     String[] expectedTexts = {
55         "To include good practices\\nand ideas from successful\\
56         nEPAM project",
57         "To be flexible and\\ncustomizable",
58         "To be multiplatform",
59         "Already have good base\\n(about 20 internal and some
60         external projects),\\nwish to get more..."
61     };
62
63     for (int i = 0; i < texts.size(); i++) {
64         softAssert.assertEquals(texts.get(i).getText(),
65             expectedTexts[i]);
66     }
67
68     // 8. Assert that there is the iframe with "Frame Button" exist
69     WebElement iframe = driver.findElement(By.id("frame"));
70     softAssert.assertTrue(iframe.isDisplayed());
71
72     // 9. Switch to the iframe and check that there is "Frame Button
73     // " in the iframe
74     driver.switchTo().frame(iframe);
75     WebElement frameButton = driver.findElement(By.id("frame-button
76         "));
77     softAssert.assertTrue(frameButton.isDisplayed());
78
79     // 10. Switch to original window back
80     driver.switchTo().defaultContent();

```

```

71
72 // 11. Assert that there are 5 items in the Left Section are
    displayed, and they have proper text
73 List<WebElement> leftMenuItems = driver.findElements(By.
    cssSelector("ul.sidebar-menu.left > li"));
74 softAssert.assertEquals(leftMenuItems.size(), 5);
75
76 String[] expectedMenuTexts = {"Home", "Contact form", "Service",
    "Metals & Colors", "Elements packs"};
77 for (int i=0; i < leftMenuItems.size(); i++) {
78     WebElement item = leftMenuItems.get(i);
79     softAssert.assertTrue(item.isDisplayed());
80     softAssert.assertEquals(item.getText(), expectedMenuTexts[i
        ]);
81 }
82
83 softAssert.assertAll();
84
85 }
86 }

```

Приложение 7. Исходный код Task1Test

```

1  import org.openqa.selenium.By;
2  import org.openqa.selenium.WebElement;
3  import org.testng.annotations.Test;
4  import org.testng.asserts.SoftAssert;
5
6  import java.util.Arrays;
7  import java.util.List;
8
9  import static org.testng.Assert.assertEquals;
10
11 public class Task2Test extends DriverSetup {
12
13     @Test
14     public void testTitle() {
15         // 1. Проверить заголовок страницы
16         driver.get("https://jdi-testing.github.io/jdi-light/index.
            html");
17         assertEquals(driver.getTitle(), "Home Page");
18     }
19
20     @Test
21     public void testLogin() {
22         // 2. Выполнить логин пользователя
23         driver.findElement(By.id("user-icon")).click();
24         driver.findElement(By.id("name")).sendKeys("Roman");
25         driver.findElement(By.id("password")).sendKeys("Jdi1234");
26         driver.findElement(By.id("login-button")).click();
27         assertEquals(driver.findElement(By.id("user-name")).getText
            (), "ROMAN IOVLEV");
28     }
29
30     @Test
31     public void testElement() {
32         // 3. Перейти на страницу с чекбоксами, радиокнопками и выпа
            дающими списками

```

```

33     driver.findElement(By.cssSelector("body>header>div>nav>ul.
        uui-navigation.nav.navbar-nav.m-18>li>a>span")).click();
34     driver.findElement(By.xpath("/html/body/header/div/nav/ul
        [1]/li[3]/ul/li[8]/a")).click();
35
36     // 4. Выбрать чекбоксы "Water" и "Wind"
37     List<WebElement> checkboxes = driver.findElements(By.
        className("label-checkbox"));
38     for (WebElement checkbox : checkboxes) {
39         if (checkbox.getText().equals("Water") || checkbox.
            getText().equals("Wind")) {
40             checkbox.click();
41         }
42     }
43     // 5. Выбрать радиокнопку "Selen"
44     List<WebElement> radios = driver.findElements(By.className("
        label-radio"));
45     for (WebElement radio : radios) {
46         if (radio.getText().equals("Selen")) {
47             radio.click();
48         }
49     }
50
51     // 6. Выбрать опцию "Yellow" из выпадающего списка
52     List<WebElement> options = driver.findElements(By.tagName("
        option"));
53     for (WebElement option : options) {
54         if (option.getText().equals("Yellow")) {
55             option.click();
56         }
57     }
58
59     // 7. Проверить логи изменений
60     final int logIndexStart = 9;
61     String logsText = driver.findElement(By.cssSelector("ul.
        panel-body-list.logs")).getText();
62     String[] logLinesArray = logsText.split("\n");
63     List<String> logLines = Arrays.stream(logLinesArray).map(log
        -> log.substring(logIndexStart)).toList();
64
65     // Ожидаемые записи в логах
66     List<String> expectedLogEntries = List.of(
67         "Цвета: value changed to Yellow",
68         "металл: value changed to Selen",
69         "Берег: condition changed to true",
70         "Вода: condition changed to true"
71     );
72
73     // Проверить соответствие логов
74     assertEquals(logLines, expectedLogEntries);
75
76     }
77 }

```